

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df=pd.read_csv('Shark Tank US dataset.csv')
#STEP 1: Understand the Dataset
df.head()
```

	Season Number	Startup Name	Episode Number	Pitch
Number \				
0	1	AvaTheElephant	1	
1				
1	1	MrTod'sPieFactory	1	
2				
2	1	Wispots	1	
3				
3	1	CollegeFoxesPackingBoxes	1	
4				
4	1	IonicEar	1	
5				

	Season Start	Season End	Original Air Date	Industry \
0	9-Aug-09	5-Feb-10	9-Aug-09	Health/Wellness
1	9-Aug-09	5-Feb-10	9-Aug-09	Food and Beverage
2	9-Aug-09	5-Feb-10	9-Aug-09	Business Services
3	9-Aug-09	5-Feb-10	9-Aug-09	Lifestyle/Home
4	9-Aug-09	5-Feb-10	9-Aug-09	Technology/Software

	Business Description \
0	Ava The Elephant - Baby and Child Care
1	Mr. Tod's Pie Factory - Specialty Food
2	Wispots - Consumer Services
3	College Foxes Packing Boxes - Consumer Services
4	Ionic Ear - Novelties

	Company Website	... Guest Investment Amount \
0	http://www.avathee elephant.com/	NaN
1	http://whybake.com/	NaN
2	http://www.wispots.com/	NaN
3	http://collegehunkshaulingjunk.com/	NaN
4	NaN	NaN

	Guest Investment Equity	Guest Name	Barbara Corcoran Present \
0	NaN	NaN	1.0
1	NaN	NaN	1.0
2	NaN	NaN	1.0
3	NaN	NaN	1.0
4	NaN	NaN	1.0

	Mark Cuban Present	Lori Greiner Present	Robert Herjavec Present	\
0	0.0	0.0	1.0	
1	0.0	0.0	1.0	
2	0.0	0.0	1.0	
3	0.0	0.0	1.0	
4	0.0	0.0	1.0	

	Daymond John Present	Kevin O Leary Present	Guest Present
0	1.0	1.0	NaN
1	1.0	1.0	NaN
2	1.0	1.0	NaN
3	1.0	1.0	NaN
4	1.0	1.0	NaN

[5 rows x 53 columns]

df.shape

(1485, 54)

df.info()

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1485 entries, 0 to 1484

Data columns (total 53 columns):

#	Column	Non-Null Count	Dtype
0	Season Number	1485 non-null	int64
1	Startup Name	1485 non-null	object
2	Episode Number	1485 non-null	int64
3	Pitch Number	1485 non-null	int64
4	Season Start	1485 non-null	object
5	Season End	1441 non-null	object
6	Original Air Date	1485 non-null	object
7	Industry	1485 non-null	object
8	Business Description	1485 non-null	object
9	Company Website	961 non-null	object
10	Pitchers Gender	1476 non-null	object
11	Pitchers Average Age	547 non-null	object
12	Pitchers City	818 non-null	object
13	Pitchers State	946 non-null	object
14	Entrepreneur Names	986 non-null	object
15	Multiple Entrepreneurs	1056 non-null	float64
16	US Viewership	1473 non-null	float64
17	Original Ask Amount	1481 non-null	float64
18	Original Offered Equity	1481 non-null	float64
19	Valuation Requested	1481 non-null	float64
20	Got Deal	1481 non-null	float64
21	Total Deal Amount	914 non-null	float64
22	Total Deal Equity	914 non-null	float64

23	Deal Valuation	914 non-null	float64
24	Number of Sharks in Deal	914 non-null	float64
25	Investment Amount Per Shark	914 non-null	float64
26	Equity Per Shark	914 non-null	float64
27	Royalty Deal	93 non-null	object
28	Advisory Shares Equity	6 non-null	float64
29	Loan	60 non-null	float64
30	Deal Has Conditions	4 non-null	object
31	Barbara Corcoran Investment Amount	146 non-null	float64
32	Barbara Corcoran Investment Equity	146 non-null	float64
33	Mark Cuban Investment Amount	263 non-null	float64
34	Mark Cuban Investment Equity	263 non-null	float64
35	Lori Greiner Investment Amount	240 non-null	float64
36	Lori Greiner Investment Equity	240 non-null	float64
37	Robert Herjavec Investment Amount	138 non-null	float64
38	Robert Herjavec Investment Equity	138 non-null	float64
39	Daymond John Investment Amount	126 non-null	float64
40	Daymond John Investment Equity	126 non-null	float64
41	Kevin O Leary Investment Amount	138 non-null	float64
42	Kevin O Leary Investment Equity	138 non-null	float64
43	Guest Investment Amount	153 non-null	float64
44	Guest Investment Equity	153 non-null	float64
45	Guest Name	153 non-null	object
46	Barbara Corcoran Present	1006 non-null	float64
47	Mark Cuban Present	1064 non-null	float64
48	Lori Greiner Present	1112 non-null	float64
49	Robert Herjavec Present	988 non-null	float64
50	Daymond John Present	998 non-null	float64
51	Kevin O Leary Present	1109 non-null	float64
52	Guest Present	147 non-null	float64

dtypes: float64(35), int64(3), object(15)

memory usage: 615.0+ KB

#STEP 2: Data Quality Assessment

```
df.isnull().sum().sort_values(ascending=False)
```

Deal Has Conditions	1481
Advisory Shares Equity	1479
Loan	1425
Royalty Deal	1392
Daymond John Investment Equity	1359
Daymond John Investment Amount	1359
Kevin O Leary Investment Equity	1347
Kevin O Leary Investment Amount	1347
Robert Herjavec Investment Equity	1347
Robert Herjavec Investment Amount	1347
Barbara Corcoran Investment Equity	1339
Barbara Corcoran Investment Amount	1339
Guest Present	1338

Guest Name	1332
Guest Investment Amount	1332
Guest Investment Equity	1332
Lori Greiner Investment Equity	1245
Lori Greiner Investment Amount	1245
Mark Cuban Investment Equity	1222
Mark Cuban Investment Amount	1222
Pitchers Average Age	938
Pitchers City	667
Investment Amount Per Shark	571
Equity Per Shark	571
Total Deal Equity	571
Number of Sharks in Deal	571
Total Deal Amount	571
Deal Valuation	571
Pitchers State	539
Company Website	524
Entrepreneur Names	499
Robert Herjavec Present	497
Daymond John Present	487
Barbara Corcoran Present	479
Multiple Entrepreneurs	429
Mark Cuban Present	421
Kevin O Leary Present	376
Lori Greiner Present	373
Season End	44
US Viewership	12
Pitchers Gender	9
Original Offered Equity	4
Got Deal	4
Original Ask Amount	4
Valuation Requested	4
Season Number	0
Startup Name	0
Business Description	0
Industry	0
Original Air Date	0
Season Start	0
Pitch Number	0
Episode Number	0
Guest_Present_Binary	0
dtype: int64	

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df['Original Air Date']=pd.to_datetime(df['Original Air Date'])
```

```
C:\Users\thesi\AppData\Local\Temp\ipykernel_2524\1486867306.py:1:
UserWarning: Could not infer format, so each element will be parsed
individually, falling back to `dateutil`. To ensure parsing is
consistent and as-expected, please specify a format.
```

```
df['Original Air Date']=pd.to_datetime(df['Original Air Date'])
```

```
df['Original Air Date'].head(10)
```

```
0    2009-08-09
1    2009-08-09
2    2009-08-09
3    2009-08-09
4    2009-08-09
5    2009-08-16
6    2009-08-16
7    2009-08-16
8    2009-08-16
9    2009-08-16
```

```
Name: Original Air Date, dtype: datetime64[ns]
```

```
df['Original Air Date']=pd.to_datetime(df['Original Air
Date'],format='%y-%m-%d')
```

```
df.info()    #now change datetime
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1485 entries, 0 to 1484
Data columns (total 53 columns):
```

#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	Season Number	1485 non-null	int64
1	Startup Name	1485 non-null	object
2	Episode Number	1485 non-null	int64
3	Pitch Number	1485 non-null	int64
4	Season Start	1485 non-null	object
5	Season End	1441 non-null	object
6	Original Air Date	1485 non-null	datetime64[ns]
7	Industry	1485 non-null	object
8	Business Description	1485 non-null	object
9	Company Website	961 non-null	object

10	Pitchers Gender	1476 non-null	object
11	Pitchers Average Age	547 non-null	object
12	Pitchers City	818 non-null	object
13	Pitchers State	946 non-null	object
14	Entrepreneur Names	986 non-null	object
15	Multiple Entrepreneurs	1056 non-null	float64
16	US Viewership	1473 non-null	float64
17	Original Ask Amount	1481 non-null	float64
18	Original Offered Equity	1481 non-null	float64
19	Valuation Requested	1481 non-null	float64
20	Got Deal	1481 non-null	float64
21	Total Deal Amount	914 non-null	float64
22	Total Deal Equity	914 non-null	float64
23	Deal Valuation	914 non-null	float64
24	Number of Sharks in Deal	914 non-null	float64
25	Investment Amount Per Shark	914 non-null	float64
26	Equity Per Shark	914 non-null	float64
27	Royalty Deal	93 non-null	object
28	Advisory Shares Equity	6 non-null	float64
29	Loan	60 non-null	float64
30	Deal Has Conditions	4 non-null	object
31	Barbara Corcoran Investment Amount	146 non-null	float64
32	Barbara Corcoran Investment Equity	146 non-null	float64
33	Mark Cuban Investment Amount	263 non-null	float64
34	Mark Cuban Investment Equity	263 non-null	float64
35	Lori Greiner Investment Amount	240 non-null	float64

36	Lori Greiner Investment Equity	240	non-null	float64
37	Robert Herjavec Investment Amount	138	non-null	float64
38	Robert Herjavec Investment Equity	138	non-null	float64
39	Daymond John Investment Amount	126	non-null	float64
40	Daymond John Investment Equity	126	non-null	float64
41	Kevin O Leary Investment Amount	138	non-null	float64
42	Kevin O Leary Investment Equity	138	non-null	float64
43	Guest Investment Amount	153	non-null	float64
44	Guest Investment Equity	153	non-null	float64
45	Guest Name	153	non-null	object
46	Barbara Corcoran Present	1006	non-null	float64
47	Mark Cuban Present	1064	non-null	float64
48	Lori Greiner Present	1112	non-null	float64
49	Robert Herjavec Present	988	non-null	float64
50	Daymond John Present	998	non-null	float64
51	Kevin O Leary Present	1109	non-null	float64
52	Guest Present	147	non-null	float64

dtypes: datetime64[ns](1), float64(35), int64(3), object(14)
memory usage: 615.0+ KB

```
print(df['Original Air Date'].sample(20).tolist()) # why this check
because there are different type of format in columns
```

```
[Timestamp('2020-03-06 00:00:00'), Timestamp('2022-03-25 00:00:00'),
Timestamp('2012-03-16 00:00:00'), Timestamp('2011-04-29 00:00:00'),
Timestamp('2025-09-24 00:00:00'), Timestamp('2013-11-15 00:00:00'),
Timestamp('2019-04-21 00:00:00'), Timestamp('2022-05-06 00:00:00'),
Timestamp('2015-03-13 00:00:00'), Timestamp('2013-09-20 00:00:00'),
Timestamp('2026-01-14 00:00:00'), Timestamp('2024-03-15 00:00:00'),
Timestamp('2020-03-27 00:00:00'), Timestamp('2025-05-09 00:00:00'),
Timestamp('2013-01-04 00:00:00'), Timestamp('2023-01-27 00:00:00'),
Timestamp('2017-11-05 00:00:00'), Timestamp('2018-01-21 00:00:00'),
Timestamp('2014-09-26 00:00:00'), Timestamp('2013-09-20 00:00:00')]
```

```

df['Original Air Date'].unique()[:20]
<DatetimeArray>
['2009-08-09 00:00:00', '2009-08-16 00:00:00', '2009-08-23 00:00:00',
 '2009-08-30 00:00:00', '2009-09-06 00:00:00', '2009-09-13 00:00:00',
 '2009-09-29 00:00:00', '2009-10-06 00:00:00', '2009-10-13 00:00:00',
 '2009-10-20 00:00:00', '2010-01-08 00:00:00', '2010-01-15 00:00:00',
 '2010-01-29 00:00:00', '2010-02-05 00:00:00', '2011-03-20 00:00:00',
 '2011-03-25 00:00:00', '2011-04-01 00:00:00', '2011-04-08 00:00:00',
 '2011-04-15 00:00:00', '2011-04-22 00:00:00']
Length: 20, dtype: datetime64[ns]

df.duplicated().sum()
np.int64(0)

df['Guest Present'].head(10)
0    NaN
1    NaN
2    NaN
3    NaN
4    NaN
5    NaN
6    NaN
7    NaN
8    NaN
9    NaN
Name: Guest Present, dtype: float64

df['Guest Present'].tail(10)
1475    2.0
1476    2.0
1477    2.0
1478    2.0
1479    2.0
1480    2.0
1481    2.0
1482    2.0
1483    2.0
1484    2.0
Name: Guest Present, dtype: float64

df['Guest Present'].sample(10) # so there are three value in guest
present so convert into int
#and remove Nan values first verify actual mean of Nan,1 and 2
1070    NaN
784     NaN
1246    NaN

```



```
1186    NaN
467     NaN
36      NaN
90      NaN
1314    NaN
1171    NaN
817     NaN
```

```
Name: Guest Present, dtype: float64
```

```
df['Guest Present'].value_counts(dropna=False)
```

```
Guest Present
```

```
NaN      1338
1.0       123
2.0        24
```

```
Name: count, dtype: int64
```

```
df['Guest_Present_Binary'] = df['Guest
Present'].notna().astype('int8') #so its basically convert nan=0 and
1=1,2=1 that mean 0=no gest and 1=present guest
```

```
df.info()
```

```
df['Guest_Present_Binary'].head(10)
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1485 entries, 0 to 1484
```

```
Data columns (total 63 columns):
```

#	Column	Non-Null Count	Dtype
0	Season Number	1485 non-null	int64
1	Startup Name	1485 non-null	object
2	Episode Number	1485 non-null	int64
3	Pitch Number	1485 non-null	int64
4	Season Start	1485 non-null	object
5	Season End	1441 non-null	object
6	Original Air Date	1485 non-null	datetime64[ns]
7	Industry	1485 non-null	int64
8	Business Description	1485 non-null	object
9	Company Website	961 non-null	object

10	Pitchers Gender	1476 non-null	object
11	Pitchers Average Age	547 non-null	object
12	Pitchers City	818 non-null	object
13	Entrepreneur Names	986 non-null	object
14	Multiple Entrepreneurs	1056 non-null	float64
15	US Viewership	1473 non-null	float64
16	Original Ask Amount	1481 non-null	float64
17	Original Offered Equity	1481 non-null	float64
18	Valuation Requested	1481 non-null	float64
19	Got Deal	1481 non-null	float64
20	Total Deal Amount	914 non-null	float64
21	Total Deal Equity	914 non-null	float64
22	Deal Valuation	914 non-null	float64
23	Number of Sharks in Deal	914 non-null	float64
24	Investment Amount Per Shark	914 non-null	float64
25	Equity Per Shark	914 non-null	float64
26	Royalty Deal	93 non-null	object
27	Advisory Shares Equity	6 non-null	float64
28	Loan	60 non-null	float64
29	Deal Has Conditions	4 non-null	object
30	Barbara Corcoran Investment Amount	146 non-null	float64
31	Barbara Corcoran Investment Equity	146 non-null	float64
32	Mark Cuban Investment Amount	263 non-null	float64
33	Mark Cuban Investment Equity	263 non-null	float64
34	Lori Greiner Investment Amount	240 non-null	float64
35	Lori Greiner Investment Equity	240 non-null	float64

36	Robert Herjavec Investment Amount	138	non-null	float64
37	Robert Herjavec Investment Equity	138	non-null	float64
38	Daymond John Investment Amount	126	non-null	float64
39	Daymond John Investment Equity	126	non-null	float64
40	Kevin O Leary Investment Amount	138	non-null	float64
41	Kevin O Leary Investment Equity	138	non-null	float64
42	Guest Investment Amount	153	non-null	float64
43	Guest Investment Equity	153	non-null	float64
44	Guest Name	153	non-null	object
45	Barbara Corcoran Present	1006	non-null	float64
46	Mark Cuban Present	1064	non-null	float64
47	Lori Greiner Present	1112	non-null	float64
48	Robert Herjavec Present	988	non-null	float64
49	Daymond John Present	998	non-null	float64
50	Kevin O Leary Present	1109	non-null	float64
51	Guest Present	147	non-null	float64
52	Guest_Present_Binary	1485	non-null	int8
53	Pitchers State_CA	1485	non-null	bool
54	Pitchers State_CO	1485	non-null	bool
55	Pitchers State_FL	1485	non-null	bool
56	Pitchers State_GA	1485	non-null	bool
57	Pitchers State_IL	1485	non-null	bool
58	Pitchers State_NY	1485	non-null	bool
59	Pitchers State_OR	1485	non-null	bool
60	Pitchers State_Other	1485	non-null	bool
61	Pitchers State_TX	1485	non-null	bool

```
62 Pitches State_UT 1485 non-null bool
```

```
dtypes: bool(10), datetime64[ns](1), float64(35), int64(4), int8(1),  
object(12)
```

```
memory usage: 619.4+ KB
```

```
0    0  
1    0  
2    0  
3    0  
4    0  
5    0  
6    0  
7    0  
8    0  
9    0
```

```
Name: Guest_Present_Binary, dtype: int8
```

```
#Categorize Features
```

```
df['Industry'] = le.fit_transform(df['Industry'])
```

```
df['Industry'].sample(5)
```

```
651    2  
938    6  
497    6  
564    9  
1124   6
```

```
Name: Industry, dtype: int64
```

```
print(df.columns.tolist())
```

```
['Season Number', 'Startup Name', 'Episode Number', 'Pitch Number',  
'Season Start', 'Season End', 'Original Air Date', 'Industry',  
'Business Description', 'Company Website', 'Pitchers Gender',  
'Pitchers Average Age', 'Entrepreneur Names', 'Multiple  
Entrepreneurs', 'US Viewership', 'Original Ask Amount', 'Original  
Offered Equity', 'Valuation Requested', 'Got Deal', 'Total Deal  
Amount', 'Total Deal Equity', 'Deal Valuation', 'Number of Sharks in  
Deal', 'Investment Amount Per Shark', 'Equity Per Shark', 'Royalty  
Deal', 'Advisory Shares Equity', 'Loan', 'Deal Has Conditions',  
'Barbara Corcoran Investment Amount', 'Barbara Corcoran Investment  
Equity', 'Mark Cuban Investment Amount', 'Mark Cuban Investment  
Equity', 'Lori Greiner Investment Amount', 'Lori Greiner Investment  
Equity', 'Robert Herjavec Investment Amount', 'Robert Herjavec  
Investment Equity', 'Daymond John Investment Amount', 'Daymond John  
Investment Equity', 'Kevin O Leary Investment Amount', 'Kevin O Leary  
Investment Equity', 'Guest Investment Amount', 'Guest Investment  
Equity', 'Guest Name', 'Barbara Corcoran Present', 'Mark Cuban
```

```
Present', 'Lori Greiner Present', 'Robert Herjavec Present', 'Daymond  
John Present', 'Kevin O Leary Present', 'Guest Present',  
'Guest_Present_Binary', 'Pitchers State_CA', 'Pitchers State_CO',  
'Pitchers State_FL', 'Pitchers State_GA', 'Pitchers State_IL',  
'Pitchers State_NY', 'Pitchers State_OR', 'Pitchers State_Other',  
'Pitchers State_TX', 'Pitchers State_UT', 'Pitchers City_Atlanta',  
'Pitchers City_Austin', 'Pitchers City_Chicago', 'Pitchers  
City_Dallas', 'Pitchers City_Denver', 'Pitchers City_Los Angeles',  
'Pitchers City_New York', 'Pitchers City_Other', 'Pitchers  
City_Portland', 'Pitchers City_San Diego', 'Pitchers City_San  
Francisco']
```

```
numerical_features = df.select_dtypes(  
    include=['int64', 'float64']  
) .columns.tolist()
```

```
categorical_features = df.select_dtypes(  
    include=['object']  
) .columns.tolist()
```

```
date_features = df.select_dtypes(  
    include=['datetime64']  
) .columns.tolist()
```

```
binary_features = []
```

```
for col in df.columns:  
    if df[col].nunique() == 2:  
        binary_features.append(col)
```

```
print("Numerical Features:")  
print(numerical_features)
```

```
print("\nCategorical Features:")  
print(categorical_features)
```

```
print("\nDate Features:")  
print(date_features)
```

```
print("\nBinary Features:")  
print(binary_features)
```

```
Numerical Features:
```

```
['Season Number', 'Episode Number', 'Pitch Number', 'Industry',  
'Multiple Entrepreneurs', 'US Viewership', 'Original Ask Amount',  
'Original Offered Equity', 'Valuation Requested', 'Got Deal', 'Total  
Deal Amount', 'Total Deal Equity', 'Deal Valuation', 'Number of Sharks  
in Deal', 'Investment Amount Per Shark', 'Equity Per Shark', 'Advisory  
Shares Equity', 'Loan', 'Barbara Corcoran Investment Amount', 'Barbara  
Corcoran Investment Equity', 'Mark Cuban Investment Amount', 'Mark  
Cuban Investment Equity', 'Lori Greiner Investment Amount', 'Lori
```

```
Greiner Investment Equity', 'Robert Herjavec Investment Amount',  
'Robert Herjavec Investment Equity', 'Daymond John Investment Amount',  
'Daymond John Investment Equity', 'Kevin O Leary Investment Amount',  
'Kevin O Leary Investment Equity', 'Guest Investment Amount', 'Guest  
Investment Equity', 'Barbara Corcoran Present', 'Mark Cuban Present',  
'Lori Greiner Present', 'Robert Herjavec Present', 'Daymond John  
Present', 'Kevin O Leary Present', 'Guest Present']
```

Categorical Features:

```
['Startup Name', 'Season Start', 'Season End', 'Business Description',  
'Company Website', 'Pitchers Gender', 'Pitchers Average Age',  
'Entrepreneur Names', 'Royalty Deal', 'Deal Has Conditions', 'Guest  
Name']
```

Date Features:

```
['Original Air Date']
```

Binary Features:

```
['Multiple Entrepreneurs', 'Got Deal', 'Barbara Corcoran Present',  
'Mark Cuban Present', 'Lori Greiner Present', 'Robert Herjavec  
Present', 'Daymond John Present', 'Kevin O Leary Present', 'Guest  
Present', 'Guest_Present_Binary', 'Pitchers State_CA', 'Pitchers  
State_CO', 'Pitchers State_FL', 'Pitchers State_GA', 'Pitchers  
State_IL', 'Pitchers State_NY', 'Pitchers State_OR', 'Pitchers  
State_Other', 'Pitchers State_TX', 'Pitchers State_UT', 'Pitchers  
City_Atlanta', 'Pitchers City_Austin', 'Pitchers City_Chicago',  
'Pitchers City_Dallas', 'Pitchers City_Denver', 'Pitchers City_Los  
Angeles', 'Pitchers City_New York', 'Pitchers City_Other', 'Pitchers  
City_Portland', 'Pitchers City_San Diego', 'Pitchers City_San  
Francisco']
```

*#if you work with small data set than you can create your dataset that
contain only important data and you can work on it*

#and also make it automate process

```
data_dictionary = pd.DataFrame({  
    'Column Name': df.columns,  
    'Data Type': df.dtypes.astype(str),  
    'Missing Values': df.isnull().sum(),  
    'Unique Values': df.nunique()  
})  
data_dictionary
```

	Column Name	Data Type	\
Season Number	Season Number	int64	
Startup Name	Startup Name	object	
Episode Number	Episode Number	int64	
Pitch Number	Pitch Number	int64	
Season Start	Season Start	object	
...	...	...	
Pitchers City_New York	Pitchers City_New York	bool	

Pitchers City_Other	Pitchers City_Other	bool
Pitchers City_Portland	Pitchers City_Portland	bool
Pitchers City_San Diego	Pitchers City_San Diego	bool
Pitchers City_San Francisco	Pitchers City_San Francisco	bool

	Missing Values	Unique Values
Season Number	0	17
Startup Name	0	1482
Episode Number	0	29
Pitch Number	0	1485
Season Start	0	17
...	...	...
Pitchers City_New York	0	2
Pitchers City_Other	0	2
Pitchers City_Portland	0	2
Pitchers City_San Diego	0	2
Pitchers City_San Francisco	0	2

[73 rows x 4 columns]

#Handle Missing Values

```
df['Pitchers Average Age'] = (
    df['Pitchers Average Age']
    .fillna(
        df['Pitchers Average Age'].median()
    )
)
```

```
df['Pitchers Average Age'].sample(10)
```

```
975      NaN
1139    Middle
1244    Middle
188      NaN
1393    Middle
510      NaN
246      NaN
1046    Young
806      NaN
223      NaN
```

Name: Pitchers Average Age, dtype: object

```
print(df['Pitchers Average Age'].value_counts(dropna=False))
```

```
Pitchers Average Age
NaN      938
Middle   492
Young     41
```

```

Old      14
Name: count, dtype: int64

df['Pitchers Average Age'] = df['Pitchers Average
Age'].fillna('Unknown')

print(df['Pitchers Average Age'].value_counts())

Pitchers Average Age
Unknown      938
Middle       492
Young        41
Old          14
Name: count, dtype: int64

age_mapping = {
    'Unknown': 0,
    'Young': 1,
    'Middle': 2,
    'Old': 3
}

df['Pitchers Average Age'] = df['Pitchers Average
Age'].map(age_mapping)

```

Detect Outliers

```
df.describe()
```

	Season Number	Episode Number	Pitch Number \
count	1485.000000	1485.000000	1485.000000
mean	9.039057	12.160269	743.000000
min	1.000000	1.000000	1.000000
25%	5.000000	6.000000	372.000000
50%	9.000000	11.000000	743.000000
75%	13.000000	18.000000	1114.000000
max	17.000000	29.000000	1485.000000
std	4.421198	7.335797	428.826888

	Original Air Date	Industry	Pitchers Average
Age \			
count	1485	1485.000000	
0.0			
mean	2018-01-30 22:24:58.181818112	6.535354	
NaN			
min	2009-08-09 00:00:00	0.000000	
NaN			
25%	2014-05-09 00:00:00	4.000000	
NaN			
50%	2017-11-05 00:00:00	6.000000	
NaN			

75%	2021-11-12 00:00:00	9.000000
NaN		
max	2026-01-28 00:00:00	15.000000
NaN		
std	NaN	3.357957
NaN		

	Multiple Entrepreneurs	US Viewership	Original Ask Amount \
count	1056.000000	1473.000000	1.481000e+03
mean	0.458333	4.829335	2.821296e+05
min	0.000000	1.370000	1.000000e+04
25%	0.000000	3.570000	1.000000e+05
50%	0.000000	4.550000	2.000000e+05
75%	1.000000	6.130000	3.430000e+05
max	1.000000	8.640000	5.000000e+06
std	0.498497	1.603174	3.446770e+05

	Original Offered Equity ...	Guest Investment Amount \
count	1481.000000 ...	1.530000e+02
mean	13.098359 ...	2.083834e+05
min	1.000000 ...	2.000000e+04
25%	8.000000 ...	7.500000e+04
50%	10.000000 ...	1.500000e+05
75%	20.000000 ...	2.500000e+05
max	100.000000 ...	1.250000e+06
std	8.397064 ...	2.001617e+05

	Guest Investment Equity	Barbara Corcoran Present	Mark Cuban Present \
count	153.000000		1006.000000
1064.000000			
mean	14.635948		0.611332
0.917293			
min	0.000000		0.000000
0.000000			
25%	8.000000		0.000000
1.000000			
50%	10.000000		1.000000
1.000000			
75%	20.000000		1.000000
1.000000			
max	100.000000		1.000000
1.000000			
std	12.141537		0.487690
0.275568			

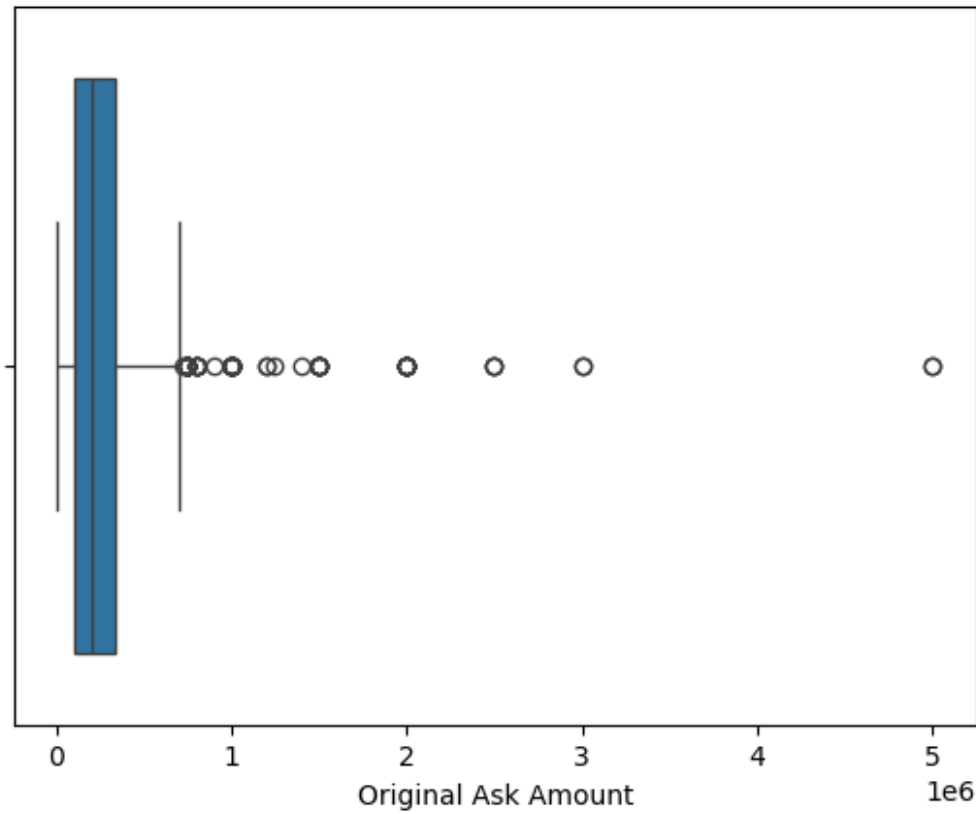
	Lori Greiner Present	Robert Herjavec Present	Daymond John Present \
count	1112.000000		988.000000
998.000000			

mean	0.798561	0.886640
0.695391		
min	0.000000	0.000000
0.000000		
25%	1.000000	1.000000
0.000000		
50%	1.000000	1.000000
1.000000		
75%	1.000000	1.000000
1.000000		
max	1.000000	1.000000
1.000000		
std	0.401256	0.317193
0.460472		

	Kevin O Leary Present	Guest Present	Guest_Present_Binary
count	1109.000000	147.000000	1485.000000
mean	0.963931	1.163265	0.098990
min	0.000000	1.000000	0.000000
25%	1.000000	1.000000	0.000000
50%	1.000000	1.000000	0.000000
75%	1.000000	1.000000	0.000000
max	1.000000	2.000000	1.000000
std	0.186545	0.370871	0.298749

[8 rows x 42 columns]

```
sns.boxplot(
    x=df['Original Ask Amount']
)
<Axes: xlabel='Original Ask Amount'>
```

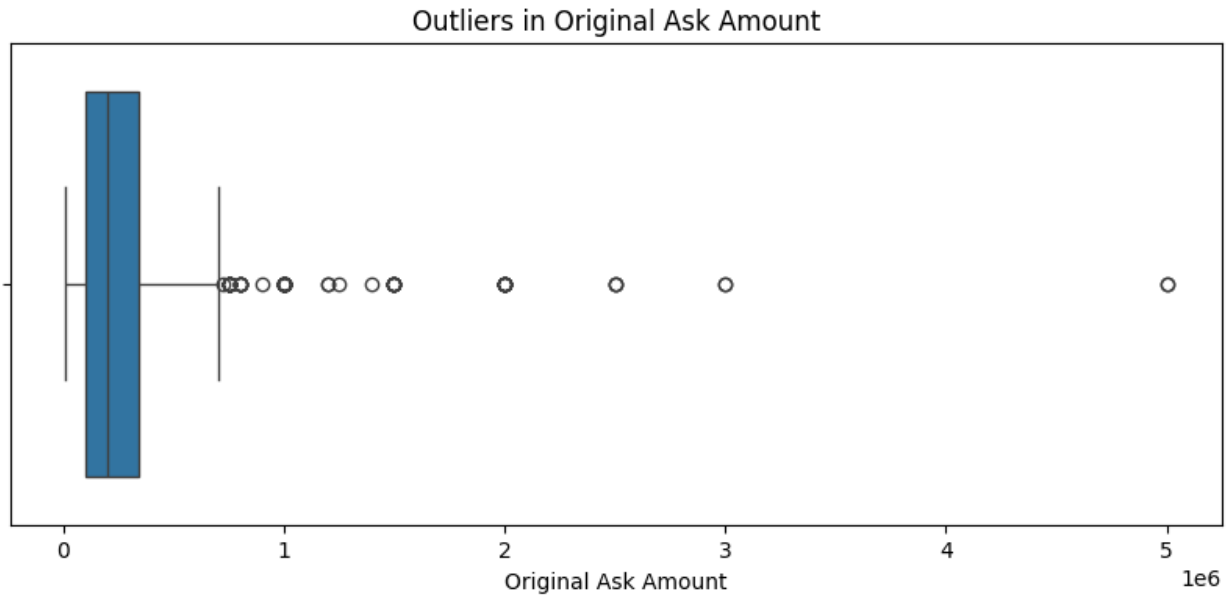


```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10,4))

sns.boxplot(
    x=df['Original Ask Amount']
)

plt.title("Outliers in Original Ask Amount")
plt.show()
```



```
Q1 = df['Original Ask Amount'].quantile(0.25)
```

```
Q3 = df['Original Ask Amount'].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
print("Q1:",Q1)
```

```
print("Q3:",Q3)
```

```
print("IQR:",IQR)
```

```
Q1: 100000.0
```

```
Q3: 343000.0
```

```
IQR: 243000.0
```

```
lower = Q1 - 1.5*IQR
```

```
upper = Q3 + 1.5*IQR
```

```
print(lower,upper)
```

```
-264500.0 707500.0
```

```
outliers = df[
    (df['Original Ask Amount'] < lower) |
    (df['Original Ask Amount'] > upper)
]
```

```
outliers
```

Season Number	Startup Name	Episode Number	Pitch Number \
1	Wisspots	1	2
1	IonicEar	1	3
			4
			5

41	1	VirtuSphere	9
42			
107	3	VegasMagicShow	2
108			
143	3	TheSullivanGenerator	11
144			
...	...	...	...
.			
1356	15	RoboBurger	21
1357			
1364	16	TRUFIT	1
1365			
1367	16	BucketGolf	2
1368			
1414	16	Blackdot	14
1415			
1443	17	Pelagion	1
1444			

	Season Start	Season End	Original Air Date	Industry \
2	9-Aug-09	5-Feb-10	2009-08-09	1
4	9-Aug-09	5-Feb-10	2009-08-09	13
41	9-Aug-09	5-Feb-10	2009-10-13	3
107	20-Jan-12	18-May-12	2012-01-27	11
143	20-Jan-12	18-May-12	2012-04-13	7
...	...	...	...	...
1356	29-Sep-23	3-May-24	2024-04-19	6
1364	18-Oct-24	16-May-25	2024-10-18	5
1367	18-Oct-24	16-May-25	2024-10-25	5
1414	18-Oct-24	16-May-25	2025-03-21	4
1443	24-Sep-25	NaN	2025-09-24	5

	Business Description	Company
Website \		
2	Wispots - Consumer Services	
	http://www.wispots.com/	
4	Ionic Ear - Novelties	
NaN		
41	Virtusphere - Electronics	
	http://www.virtusphere.com/	
107	Vegas Magic Show - Entertainment	
NaN		
143	The Sullivan Generator - Novelties	
	http://marksullivanresearch.com	
...	...	
...		
1356	Burger Chef In a Vending Format	
	https://theroboburger.com/	
1364	Custom fit sports mouthguards	

<https://trufitcustoms.com/>
 1367 Portable backyard golf game
<https://www.bucketgolfgame.com/>
 1414 Automated tattoo machine
<https://blackdot.tattoo/>
 1443 High-tech jet ski
<https://pelagion.com/>

	Pitchers City_Austin	Pitchers City_Chicago	Pitchers
City_Dallas \			
2 ...	False	False	
False			
4 ...	False	False	
False			
41 ...	False	False	
False			
107 ...	False	False	
False			
143 ...	False	False	
False			
...	...	...	...
...			
1356 ...	False	False	
False			
1364 ...	False	False	
False			
1367 ...	False	False	
False			
1414 ...	True	False	
False			
1443 ...	False	False	
False			

	Pitchers City_Denver	Pitchers City_Los Angeles	Pitchers
City_New York \			
2	False	False	
False			
4	False	False	
False			
41	False	False	
False			
107	False	False	
False			
143	False	False	
False			
...	...	...	...
...			
1356	False	False	
False			

1364		False		False
False				
1367		False		False
False				
1414		False		False
False				
1443		False		False
False				

	Pitchers	City_Other	Pitchers	City_Portland	Pitchers	City_San
Diego \						
2		True		False		
False						
4		True		False		
False						
41		True		False		
False						
107		True		False		
False						
143		True		False		
False						
...		...		...		
...						
1356		True		False		
False						
1364		True		False		
False						
1367		True		False		
False						
1414		False		False		
False						
1443		True		False		
False						

	Pitchers	City_San	Francisco
2			False
4			False
41			False
107			False
143			False
...			...
1356			False
1364			False
1367			False
1414			False
1443			False

[79 rows x 73 columns]

```
print("Number of Outliers:",len(outliers))
```

Number of Outliers: 79

```
df['Original Ask Amount']
```

```
0      50000.0
1     460000.0
2    1200000.0
3     250000.0
4   1000000.0
```

```
...
```

```
1480    250000.0
1481         NaN
1482         NaN
1483         NaN
1484         NaN
```

Name: Original Ask Amount, Length: 1485, dtype: float64

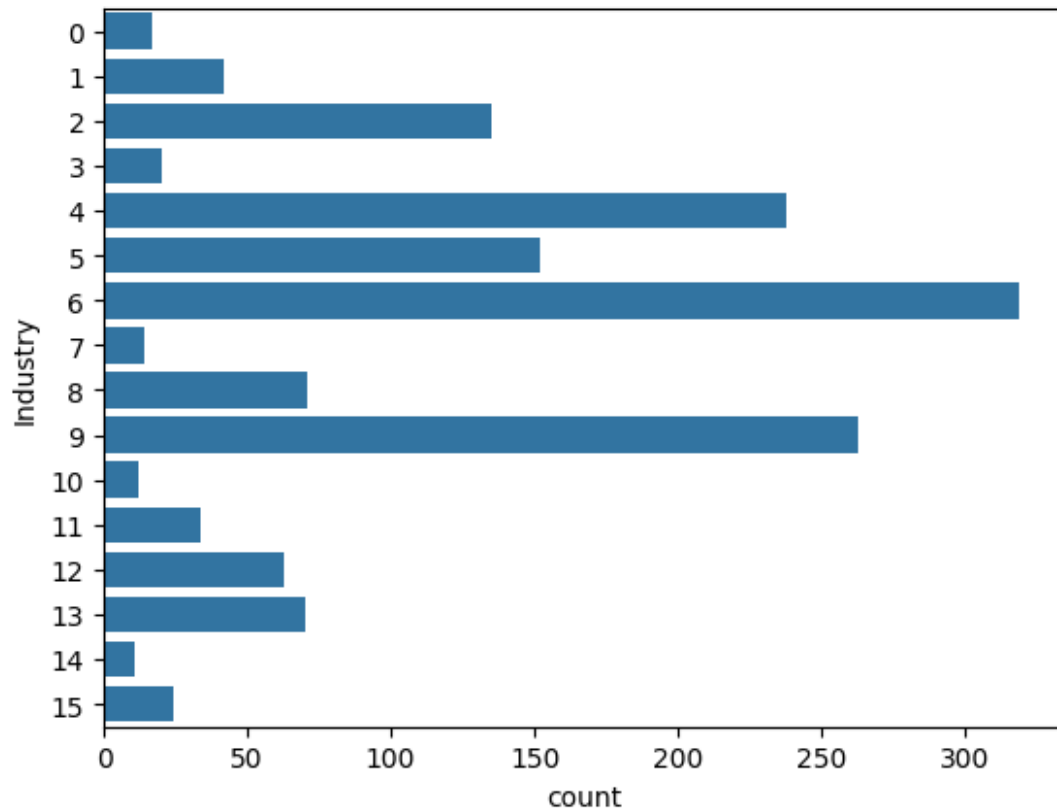
#Univariate Analysis you can deeply each column see and analysis

```
df['Industry'].value_counts() #this is category data apply it
```

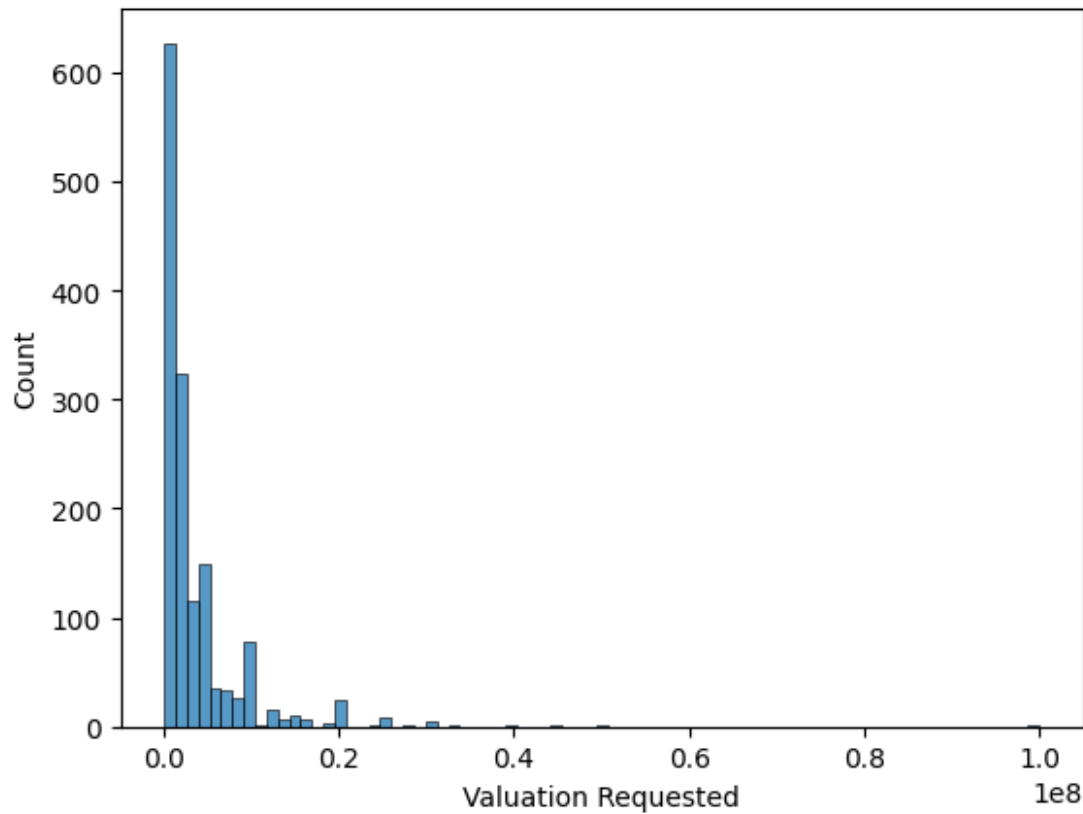
```
sns.countplot(
    y='Industry',
    data=df
```

```
)
```

```
<Axes: xlabel='count', ylabel='Industry'>
```

```
df['Valuation Requested'].describe()  
#most numeric data apply on histogram  
sns.histplot(  
    df['Valuation Requested']  
)  
  
<Axes: xlabel='Valuation Requested', ylabel='Count'>
```



#Bivariate Analysis ok so most important according to target column and depend on other column to relationship like
#two variable

```
pd.crosstab(
    df['Industry'],
    df['Got Deal']
)
```

Got Deal	0.0	1.0
Industry		
0	4	13
1	20	22
2	46	89
3	9	10
4	101	137
5	56	95
6	124	195
7	7	7
8	28	43
9	85	176
10	5	7
11	10	24
12	27	36

13	31	39
14	6	5
15	8	16

```
industry_deal = pd.crosstab(  
    df['Industry'],  
    df['Got Deal'],  
    normalize='index'  
) * 100
```

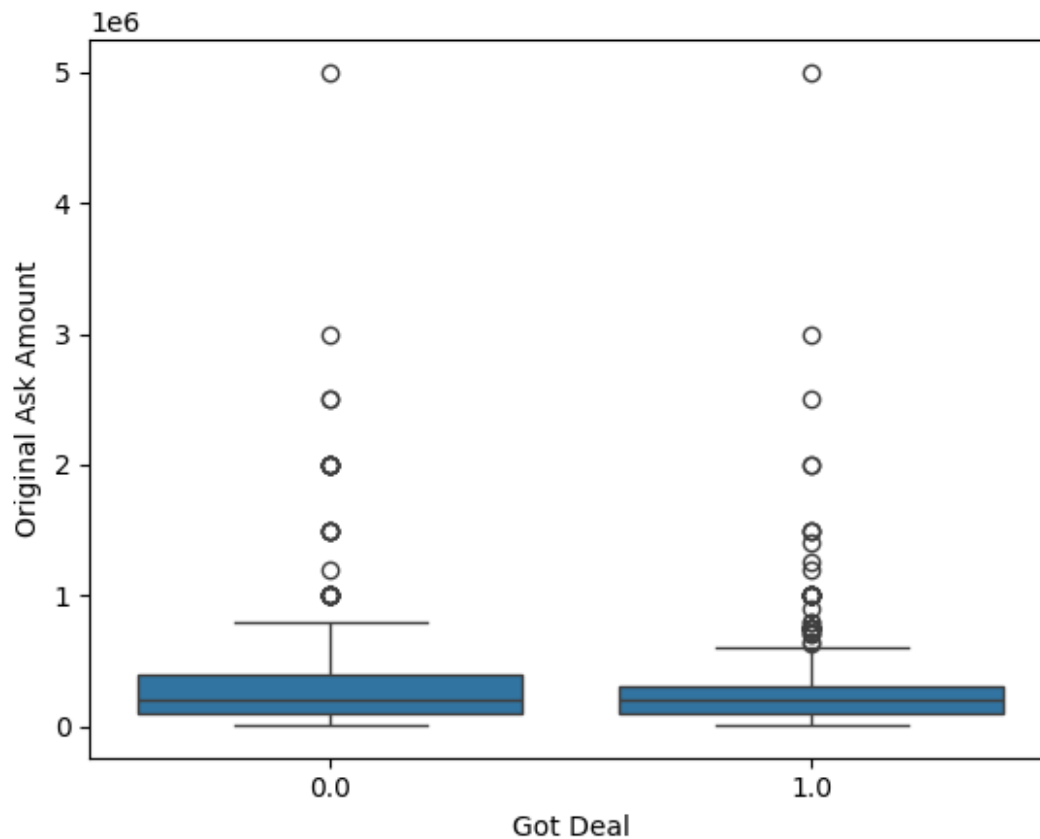
```
industry_deal.round(2)
```

Got Deal	0.0	1.0
Industry		
0	23.53	76.47
1	47.62	52.38
2	34.07	65.93
3	47.37	52.63
4	42.44	57.56
5	37.09	62.91
6	38.87	61.13
7	50.00	50.00
8	39.44	60.56
9	32.57	67.43
10	41.67	58.33
11	29.41	70.59
12	42.86	57.14
13	44.29	55.71
14	54.55	45.45
15	33.33	66.67

```
# Which industries get deals most?
```

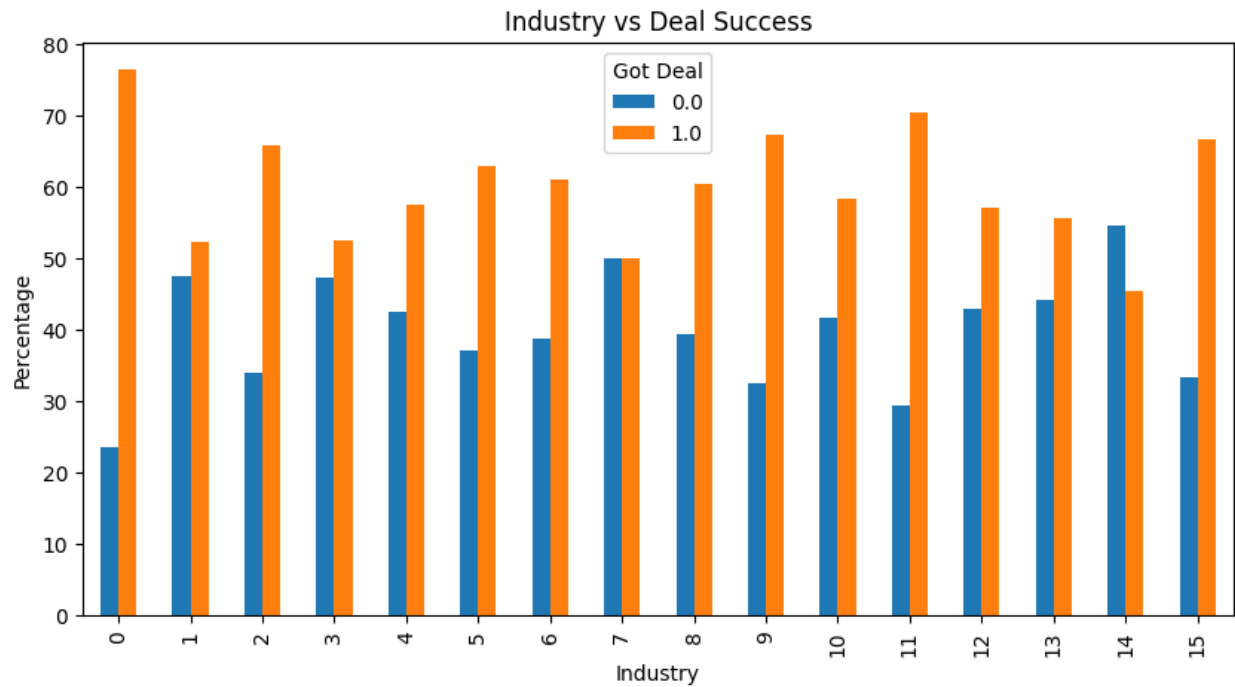
```
sns.boxplot(  
    x='Got Deal',  
    y='Original Ask Amount',  
    data=df  
)
```

```
<Axes: xlabel='Got Deal', ylabel='Original Ask Amount'>
```



```
# ok so basically you can ask industry and deal seccess rate ?
industry_deal.plot(
    kind='bar',
    figsize=(10,5)
)

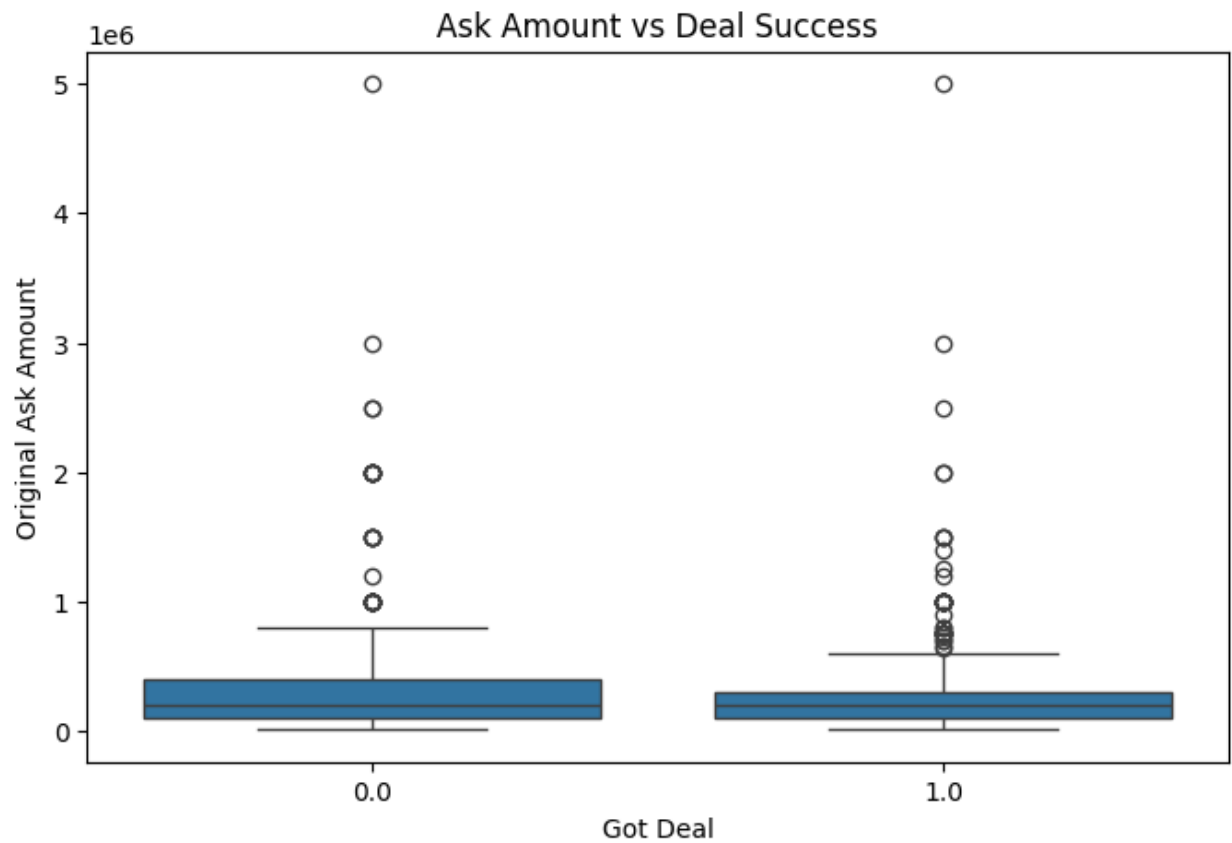
plt.title("Industry vs Deal Success")
plt.ylabel("Percentage")
plt.show()
```



```
# Do smaller asks succeed more?
plt.figure(figsize=(8,5))

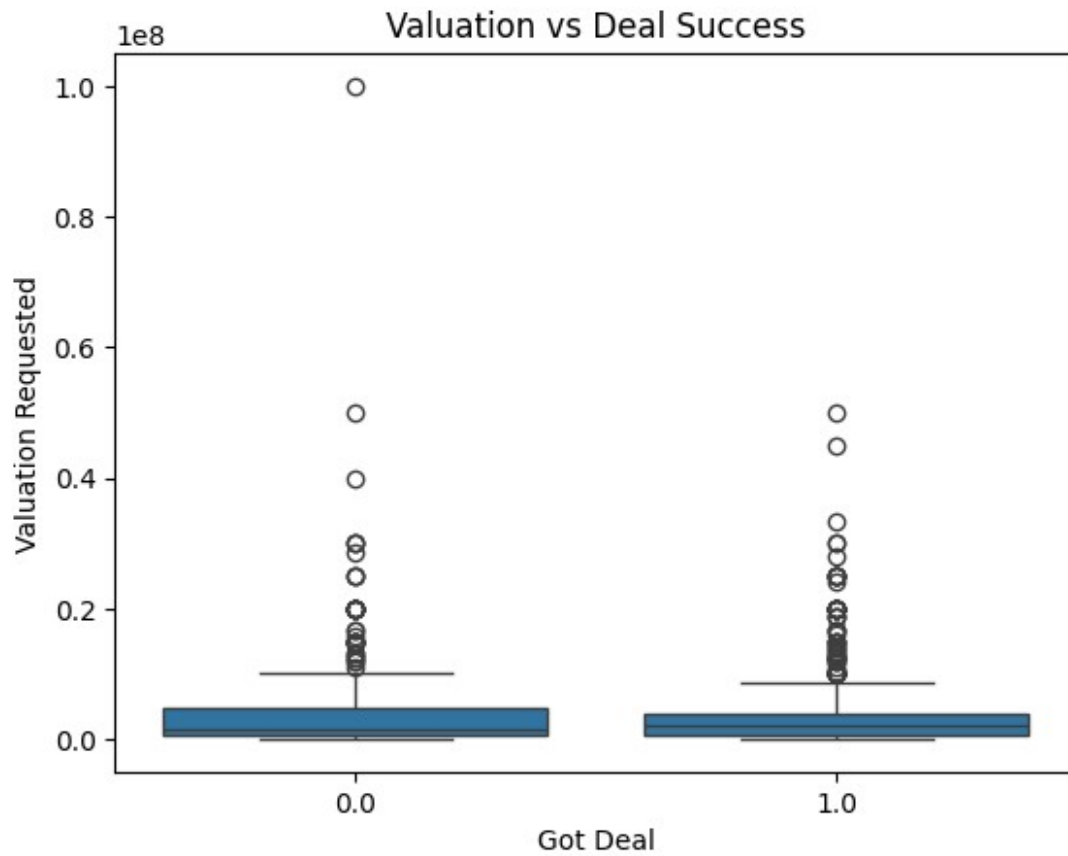
sns.boxplot(
    x='Got Deal',
    y='Original Ask Amount',
    data=df
)

plt.title("Ask Amount vs Deal Success")
plt.show()
```



```
#Valuation Requested vs Got Deal
sns.boxplot(
    x='Got Deal',
    y='Valuation Requested',
    data=df
)

plt.title("Valuation vs Deal Success")
plt.show()
```

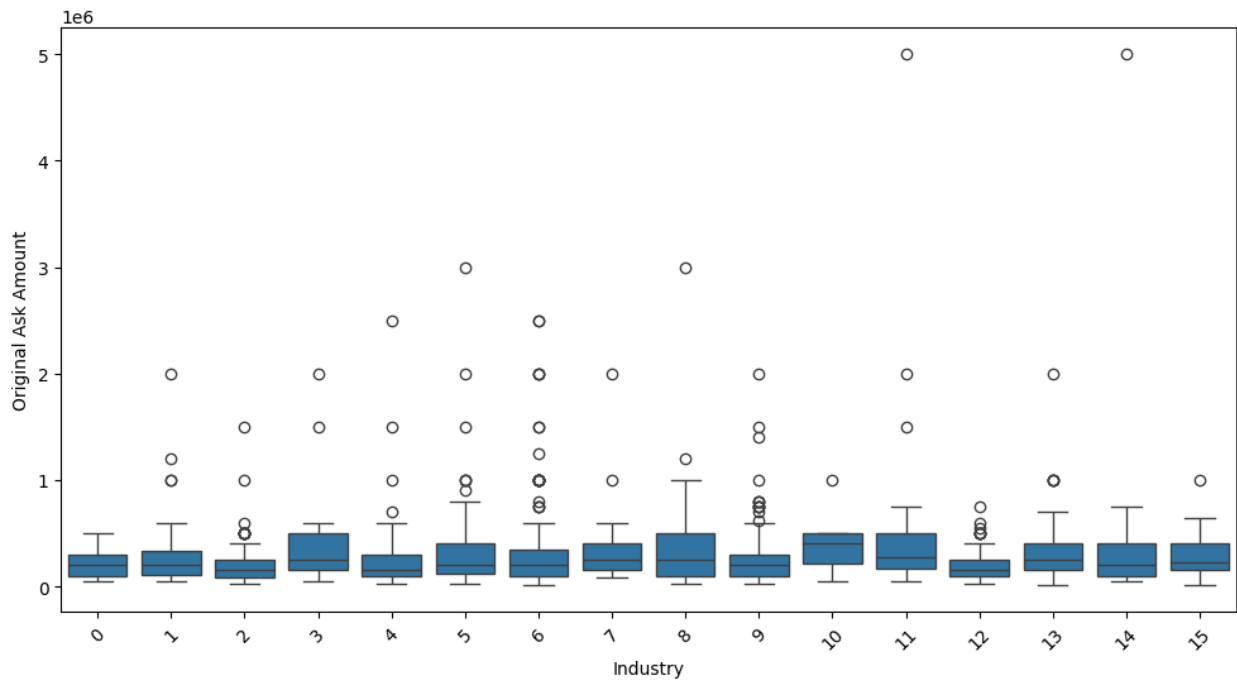


```
#Industry vs Ask Amount    you can find which industry is boom
plt.figure(figsize=(12,6))
```

```
sns.boxplot(
    x='Industry',
    y='Original Ask Amount',
    data=df
)
```

```
plt.xticks(rotation=45)
```

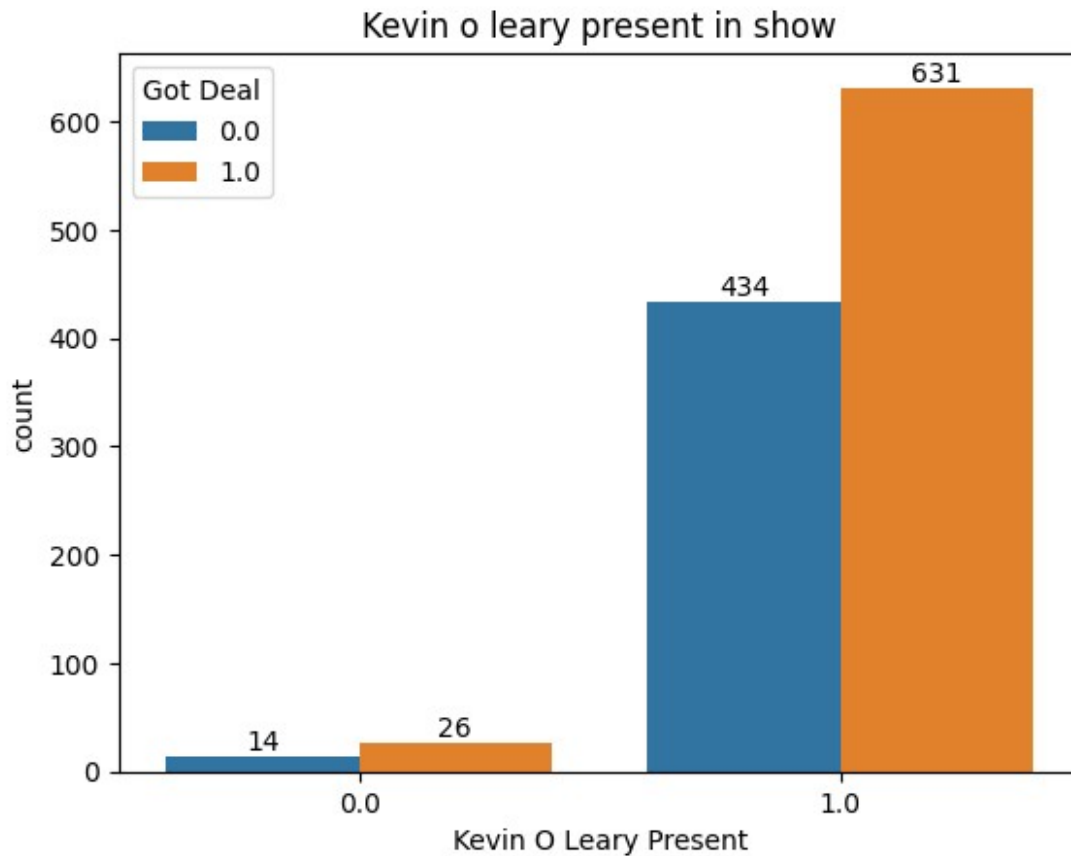
```
plt.show()
```



```
ax = sns.countplot(
    x='Kevin O Leary Present', # you can show different shark when
    # present or not their according to predict deal or not ?
    hue='Got Deal',
    data=df
)

for container in ax.containers:
    ax.bar_label(container)

plt.title('Kevin o leary present in show')
plt.show()
```

Do teams perform better than solo founders? so there are more chances if you work on team

```
pd.crosstab(
    df['Multiple Entrepreneurs'],
    df['Got Deal'],
    normalize='index'
)*100
```

1. Industry vs Got Deal

2. Ask Amount vs Got Deal

3. Valuation vs Got Deal

4. Age Group vs Got Deal

5. Multiple Entrepreneurs vs Got Deal

6. Gender vs Got Deal

7. Correlation Heatmap you can also try different columns to compare each other can relate

Got Deal	0	1
Multiple Entrepreneurs		
0	43	57
1	31	69

#Feature Engineering also try Deal Difference,Equity Change,Deal Success Flag

```
df['Valuation Requested'].head(10)
df['Valuation'] = (
    df['Original Ask Amount']
    /
    (df['Original Offered Equity'] / 100)
)
```

```
df['Valuation'].head(10)
```

```
0      333,333
1    4,600,000
2   12,000,000
3    1,000,000
4    6,666,667
5    3,333,333
6    2,500,000
7    5,000,000
8    1,000,000
9     500,000
```

```
Name: Valuation, dtype: float64
```

```
np.isinf(X).sum()    # check infinite values in dataset this is accured
in model creation
```

```
Season Number      0
Episode Number     0
Pitch Number       0
Industry           0
Pitchers Average Age 0
..
Pitchers City_San Francisco 0
Valuation           0
Equity_Ratio        0
Pitchers Gender_Male 0
Pitchers Gender_Mixed Team 0
Length: 65, dtype: int64
```

```
# "At this stage, our dataset has been fully cleaned, transformed, and
enriched through feature engineering.
```

```
# We have addressed missing values, corrected data types, handled
outliers, explored relationships between variables,
# and created new business-focused features. The resulting dataset is
now analytics-ready and can be used for advanced reporting,
# dashboard development, and predictive modeling."
```

-----next videos for create Dahboard using
PowerBi -----

